

DOCUMENTACIÓN TÉCNICA ILUSTRADA

Recicley Watermark

Marcado de agua por lotes para archivos PDF — versión con diagramas

Archivo fuente documentado: GUI_Watermark.py

Framework de interfaz: PyQt6

Plataforma: Windows 10/11 (64 bits)

Fecha del documento: 18 de julio de 2026

Índice

1. Descripción general
2. Instalación y requisitos
3. Guía de uso de la aplicación
4. Arquitectura general del código **diagrama**
5. Módulos importados y constantes globales
6. La clase `DragDropWindow` y su interfaz **diagrama**
7. Gestión del estado: lista de PDFs, marca de agua y destino
8. Diálogos de selección de archivos
9. Arrastrar y soltar (Drag & Drop)
10. `processWatermark` — flujo de decisión **diagrama**
11. `addWaterMarkToPDF` — pipeline por página **diagrama**
12. Organización de los archivos de salida **diagrama**
13. Punto de entrada `main()`
14. Empaquetado a `.exe` (PyInstaller) **diagrama**
15. Solución de problemas

1. Descripción general

Recicley Watermark es una aplicación de escritorio para Windows, construida en Python con la librería gráfica **PyQt6**, que permite aplicar una imagen de marca de agua (con transparencia) sobre uno o varios archivos PDF de forma idéntica. El usuario carga los PDF (arrastrándolos, seleccionándolos manualmente o apuntando a una carpeta completa), carga una imagen de marca de agua, ajusta la opacidad deseada y, con un solo clic, la aplicación genera copias marcadas de todos los documentos.

Internamente, cada página del PDF se convierte a una imagen de alta resolución (usando **Poppler** a través de la librería `pdf2image`), se combina con la marca de agua usando composición alfa de **Pillow** (`PIL`), y las páginas resultantes se reensamblan en un nuevo archivo PDF con **PyPDF2**.

2. Instalación y requisitos

Uso como ejecutable (recomendado)

No requiere Python ni ninguna instalación. Basta con descomprimir la carpeta `GUI_Watermark/` (que contiene `GUI_Watermark.exe` y la subcarpeta `_internal/`) y ejecutar el `.exe`. La carpeta `_internal` debe permanecer junto al ejecutable: ahí viven las DLL de PyQt6, los binarios de Poppler, el ícono de la aplicación y demás dependencias empaquetadas.

Importante: nunca copies solo el `.exe` a otra máquina sin la carpeta `_internal`; la aplicación no podrá iniciar porque le faltarán sus dependencias.

Ejecución desde el código fuente

Requisito	Uso
Python 3.13	Intérprete del lenguaje.
<code>PyQt6</code>	Interfaz gráfica (ventana, botones, slider, diálogos, drag & drop).
<code>pdf2image</code>	Convierte páginas de un PDF en imágenes, delegando en Poppler.
<code>PyPDF2</code>	Lee/escribe archivos PDF página por página.
<code>Pillow (PIL)</code>	Manipulación de imágenes: redimensionar, canal alfa, composición.
<code>poppler-0.68.0/bin</code>	Binarios nativos de Poppler que <code>pdf2image</code> invoca para renderizar.

3. Guía de uso de la aplicación

- Cargar PDF(s):** arrastra uno o varios archivos `.pdf`, o una carpeta completa (se agregan recursivamente todos los PDF que contenga), a la ventana. También puedes usar el botón **Archivos** (selector múltiple) o **Carpeta** (selector de carpeta). El botón **Limpiar** vacía la lista.
- Cargar la marca de agua:** arrastra una imagen `.png` / `.jpg` / `.jpeg` o usa el botón **Elegir imagen**. Se recomienda un PNG con canal alfa (fondo transparente) para un resultado más limpio.
- Elegir destino (opcional):** por defecto los archivos resultantes van a la carpeta *Descargas* del usuario. El botón **Elegir destino** permite cambiarla.

4. **Ajustar opacidad:** el control deslizante define qué tan visible es la marca de agua, de 0 (invisible) a 255 (totalmente opaca).
5. **Procesar:** al pulsar **Agregar Marca de Agua**, se recorre cada PDF de la lista, mostrando una barra de progreso cancelable. Al terminar se muestra un resumen y se abre automáticamente la carpeta de salida en el explorador de Windows.

4. Arquitectura general del código

Todo el proyecto vive en un único archivo, `GUI_Watermark.py`. La clase `DragDropWindow` actúa a la vez como *vista* (widgets de PyQt6) y *controlador* (lógica de negocio); el trabajo pesado de imágenes y PDF se delega a tres librerías externas. El siguiente diagrama resume cómo se relacionan estas piezas:

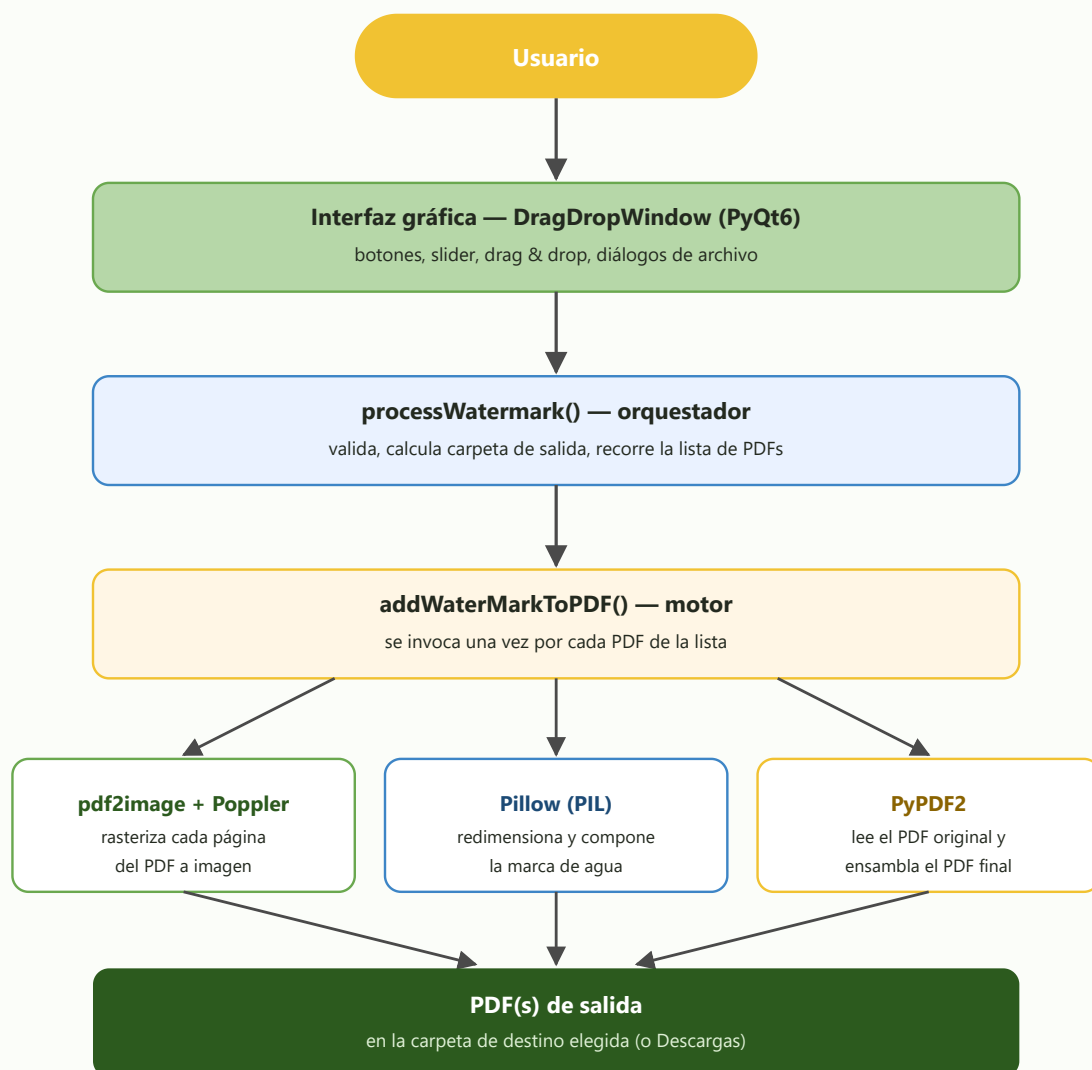


Figura 1 — Capas de la aplicación: de la interacción del usuario hasta el archivo PDF final.

5. Módulos importados y constantes globales

```
import sys
import os
import io
from datetime import datetime
from PyQt6.QtWidgets import (
    QApplication, QMainWindow, QLabel, QWidget, QPushButton, QMessageBox,
    QSlider, QFileDialog, QProgressDialog
)
from PyQt6.QtGui import QIcon
from PyQt6.QtCore import Qt

from pdf2image import convert_from_bytes
from PyPDF2 import PdfWriter, PdfReader
from PIL import Image
```

Módulo	Para qué se usa en este proyecto
<code>sys</code>	Acceso a <code>sys.argv</code> (arranque de la app Qt), <code>sys.exit()</code> y detección de si el programa corre "congelado" (empaquetado como <code>.exe</code>) vía <code>sys.frozen</code> / <code>sys._MEIPASS</code> .
<code>os</code>	Rutas de archivos y carpetas: unir rutas, listar/recorrer directorios (<code>os.walk</code>), obtener la carpeta de usuario, crear carpetas, abrir el explorador (<code>os.startfile</code>).
<code>io</code>	Buffers de memoria (<code>io.BytesIO</code>) para generar PDFs e imágenes intermedias sin escribir archivos temporales en disco.
<code>datetime</code>	Genera la marca de tiempo usada en el nombre de la subcarpeta de salida cuando se procesan varios PDFs a la vez.
<code>PyQt6.QtWidgets</code>	Todos los widgets de la interfaz: ventana principal, etiquetas, botones, slider, cuadros de diálogo de archivo/carpeta y la barra de progreso.
<code>PyQt6.QtGui.QIcon</code>	Carga el ícono de la ventana (<code>rec.jpg</code>).
<code>PyQt6.QtCore.Qt</code>	Constantes de Qt: orientación del slider, modalidad de la ventana de progreso.
<code>pdf2image.convert_from_bytes</code>	Convierte los bytes de un PDF (una página) en una imagen de Pillow, invocando internamente los binarios de Poppler.
<code>PyPDF2.PdfWriter</code> / <code>PdfReader</code>	Lectura del PDF original página por página y escritura del PDF final con las páginas ya marcadas.
<code>PIL.Image</code>	Redimensionar la marca de agua, ajustar su canal alfa según la opacidad elegida y componerla sobre cada página renderizada.

```
RENDER_DPI = 200
PDF_SAVE_QUALITY = 95
IMAGE_EXTENSIONS = ('.png', '.jpg', '.jpeg')
```

`RENDER_DPI` controla la resolución a la que Poppler renderiza cada página del PDF antes de aplicar la marca de agua. `PDF_SAVE_QUALITY` es la calidad JPEG usada por Pillow al reinsertar cada página como PDF. `IMAGE_EXTENSIONS` centraliza las extensiones aceptadas como imagen de marca de agua.

función

`get_base_path()`

```
def get_base_path():
    if getattr(sys, 'frozen', False):
        return getattr(sys, '_MEIPASS', os.path.dirname(sys.executable))
    return os.path.dirname(os.path.abspath(__file__))
```

Devuelve la carpeta desde la que deben leerse los recursos empaquetados (`rec.jpg` y `poppler-0.68.0/`): la carpeta del script en desarrollo, o `sys._MEIPASS` (la carpeta `_internal`) cuando corre como `.exe` generado por PyInstaller.

6. La clase `DragDropWindow` y su interfaz

clase

`DragDropWindow(QMainWindow)` es la única ventana de la aplicación. Su tamaño es fijo (500×520 px) y cada widget se posiciona con coordenadas absolutas (`setGeometry(x, y, ancho, alto)`). El siguiente wireframe reproduce esas coordenadas a escala, para visualizar cómo el código construye la ventana pieza por pieza:

1

Protector de archivos

2

Arrastra PDF(s) o una carpeta aquí

Archivos

Carpeta

Limpiar

3

Arrastra la imagen de marca de agua

Elegir imagen

4

Destino: predeterminado (Descargas)

Elegir destino

5

Opacidad: 128

6

Agregar Marca de Agua

Figura 2 — Wireframe de `DragDropWindow` a escala real (500×520 px), con las coordenadas exactas usadas en `setGeometry()`.

#	Atributos en el código	Rol
1	<code>title</code>	Título decorativo de la ventana.
2	<code>label_pdf</code> , <code>button_select_files</code> , <code>button_select_folder</code> , <code>button_clear_files</code>	Carga y gestión de la lista de PDF.
3	<code>label_watermark</code> , <code>button_select_watermark</code>	Carga de la imagen de marca de agua.
4	<code>label_output</code> , <code>button_select_output</code>	Carpeta de destino de los resultados.
5	<code>opacity_label</code> , <code>opacity_slider</code>	Opacidad aplicada a la marca de agua (0–255).

6	<code>button</code>	Dispara <code>processWatermark()</code> .
---	---------------------	---

7. Gestión del estado: lista de PDFs, marca de agua y destino

El estado de la aplicación vive en tres atributos de instancia, inicializados en `__init__`:

- `self.pdf_paths`: lista de rutas absolutas a los PDF cargados (puede tener 0, 1 o N elementos).
- `self.watermark_path`: ruta de la imagen de marca de agua (una sola, aplicada a todos los PDF).
- `self.output_dir`: carpeta de destino elegida por el usuario, o cadena vacía si se usará la carpeta por defecto (Descargas).

método `add_pdf(self, path)`

```
def add_pdf(self, path):
    if path not in self.pdf_paths:
        self.pdf_paths.append(path)
```

Agrega una ruta a `pdf_paths` evitando duplicados.

método `add_pdfs_from_folder(self, folder)`

```
def add_pdfs_from_folder(self, folder):
    found = []
    for root, _dirs, files in os.walk(folder):
        for f in files:
            if f.lower().endswith('.pdf'):
                found.append(os.path.join(root, f))
    for f in sorted(found):
        self.add_pdf(f)
```

`os.walk(folder)` recorre la carpeta indicada y **todas sus subcarpetas** de forma recursiva. Es la función que implementa "cargar todos los PDF de una ruta": tanto el botón **Carpeta** como arrastrar una carpeta a la ventana llaman a este mismo método.

método `update_pdf_label(self)` / `clearFiles(self)`

`update_pdf_label` sincroniza el texto de `label_pdf` (y su tooltip, con la lista completa de nombres) con el contenido real de `self.pdf_paths`, tras cualquier operación que la modifique. `clearFiles` vacía la lista y refresca la etiqueta; está conectado al botón **Limpiar**.

8. Diálogos de selección de archivos

Método	Diálogo de Qt usado	Efecto
--------	---------------------	--------

<code>selectFileDialog</code>	<code>QFileDialog.getOpenFileNames</code> (filtro <code>*.pdf</code>)	Permite elegir varios PDF a la vez; cada uno se agrega con <code>add_pdf</code> .
<code>selectFolderDialog</code>	<code>QFileDialog.getExistingDirectory</code>	Elige una carpeta y delega en <code>add_pdfs_from_folder</code> para el escaneo recursivo.
<code>selectWatermarkDialog</code>	<code>QFileDialog.getOpenFileName</code> (filtro imágenes)	Reemplaza <code>self.watermark_path</code> por la imagen elegida.
<code>selectOutputDialog</code>	<code>QFileDialog.getExistingDirectory</code>	Fija <code>self.output_dir</code> con la carpeta elegida y actualiza <code>label_output</code> .

9. Arrastrar y soltar (Drag & Drop)

método `dragEnterEvent(self, event)`

```
def dragEnterEvent(self, event):
    if event.mimeData().hasUrls():
        for url in event.mimeData().urls():
            file_path = url.toLocalFile()
            if (os.path.isdir(file_path)
                or file_path.lower().endswith('.pdf')
                or file_path.lower().endswith(IMAGE_EXTENSIONS)):
                event.acceptProposedAction()
                break
```

Qt llama a este método mientras el usuario arrastra algo sobre la ventana. Si al menos un elemento es una carpeta, un PDF o una imagen soportada, se acepta la acción.

método

dropEvent(self, event)

```
def dropEvent(self, event):
    for url in event.mimeData().urls():
        file_path = url.toLocalFile()
        if os.path.isdir(file_path):
            self.add_pdfs_from_folder(file_path)
        elif file_path.lower().endswith('.pdf'):
            self.add_pdf(file_path)
        elif file_path.lower().endswith(IMAGE_EXTENSIONS):
            self.watermark_path = file_path
            self.label_watermark.setText(...)
    self.update_pdf_label()
```

Recorre todas las URLs soltadas y las clasifica: carpeta → escaneo recursivo, PDF → se agrega a la lista, imagen → se convierte en la nueva marca de agua (la última soltada es la que queda).

10. processWatermark — flujo de decisión

Este método orquesta todo el procesamiento cuando el usuario pulsa **Agregar Marca de Agua**. El siguiente diagrama de flujo resume sus decisiones y pasos, en el mismo orden en que aparecen en el código:

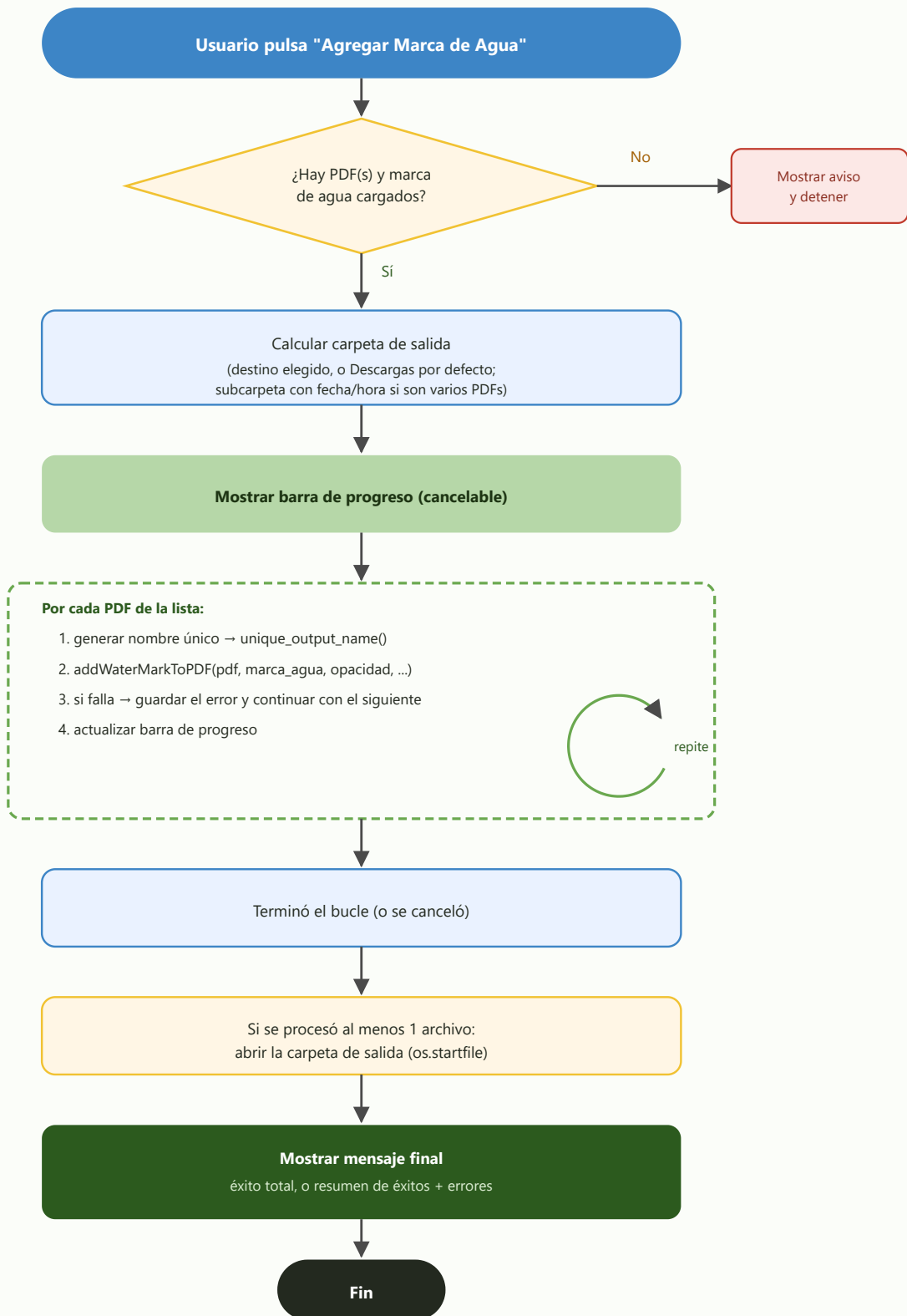


Figura 3 — Flujo completo de `processWatermark()`, desde el clic del usuario hasta el mensaje final.

11. `addWaterMarkToPDF` — pipeline por página

Este método se llama una vez por cada PDF de la lista y produce un PDF de salida completo. Dentro de él, cada página del documento pasa por una tubería de 7 pasos, repetida en un bucle:

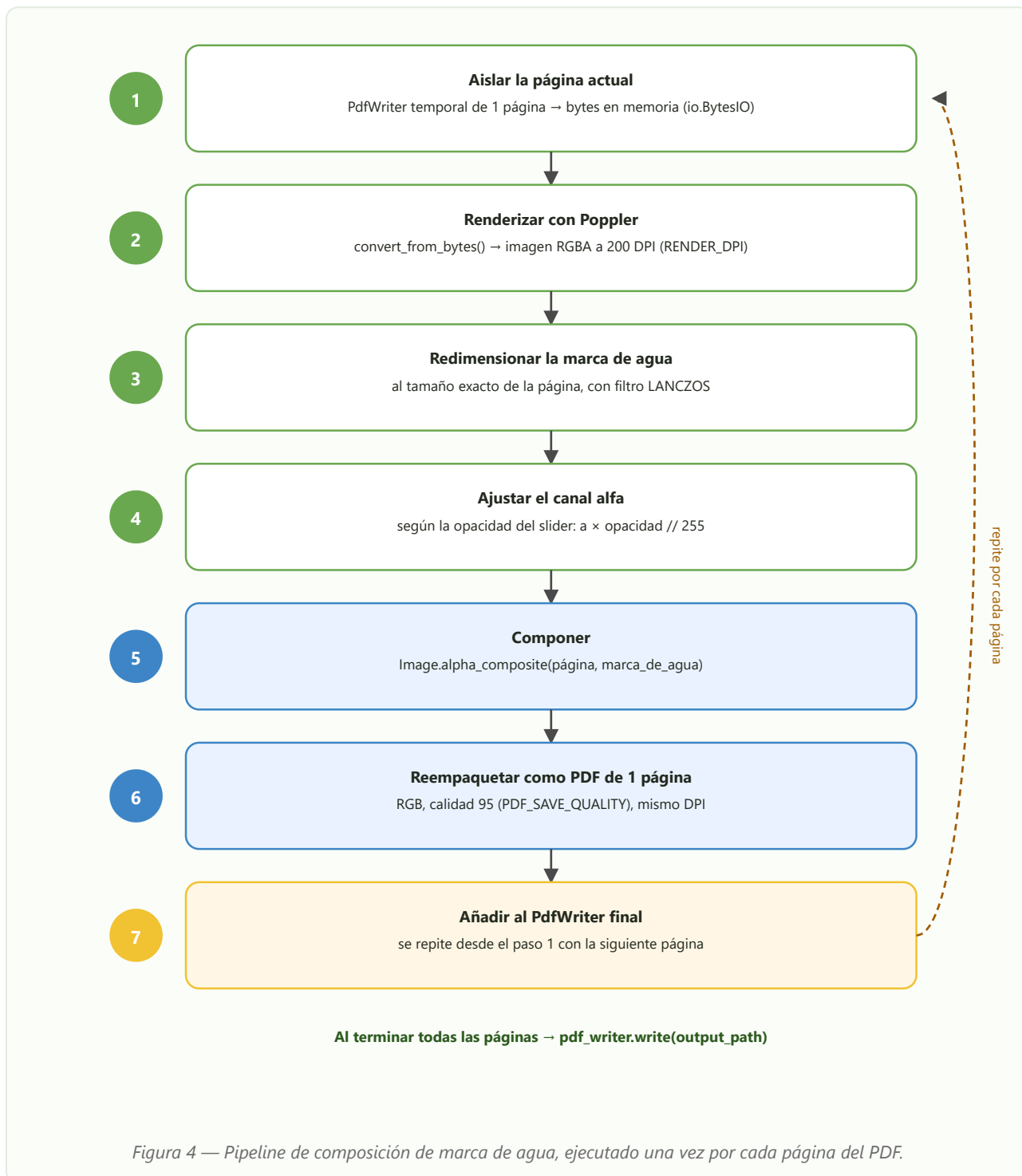


Figura 4 — Pipeline de composición de marca de agua, ejecutado una vez por cada página del PDF.

Por qué se convierte cada página a imagen en vez de dibujar la marca de agua directamente sobre el PDF vectorial: este enfoque garantiza que la marca de agua se vea idéntica sin importar el contenido original del PDF, a costa de "rasterizar" el documento — el PDF resultante ya no tiene texto seleccionable. `RENDER_DPI` y `PDF_SAVE_QUALITY` controlan el balance entre nitidez y tamaño/velocidad.

método estático`unique_output_name(used_names, filename)`

```
base, ext = os.path.splitext(filename)
candidate = filename
i = 1
while candidate in used_names:
    candidate = f"{base} ({i}){ext}"
    i += 1
used_names.add(candidate)
return candidate
```

Si el nombre propuesto ya se usó en la corrida actual, prueba `nombre (1).pdf`, `nombre (2).pdf`, etc. hasta encontrar uno libre, evitando que dos PDF de origen con igual nombre se sobrescriban.

12. Organización de los archivos de salida

El destino final depende de cuántos PDF se procesaron en la corrida:

Un solo PDF → se guarda directo en la carpeta de destino:

```
Descargas/                                     (o la carpeta elegida en "Elegir destino")
└─ informe_marca_de_agua.pdf
```

Varios PDFs → se crea una subcarpeta con fecha y hora:

```
Descargas/
└─ Recicley_Watermark_20260718_153408/
    ├── a_marca_de_agua.pdf
    ├── b_marca_de_agua.pdf
    └─ b_marca_de_agua (1).pdf      ← colisión de nombres resuelta automáticamente
                                   (por ejemplo, dos "b.pdf" en subcarpetas distintas)
```

Figura 5 — Reglas de nomenclatura y ubicación aplicadas por `processWatermark()` y `unique_output_name()`.

13. Punto de entrada `main()`

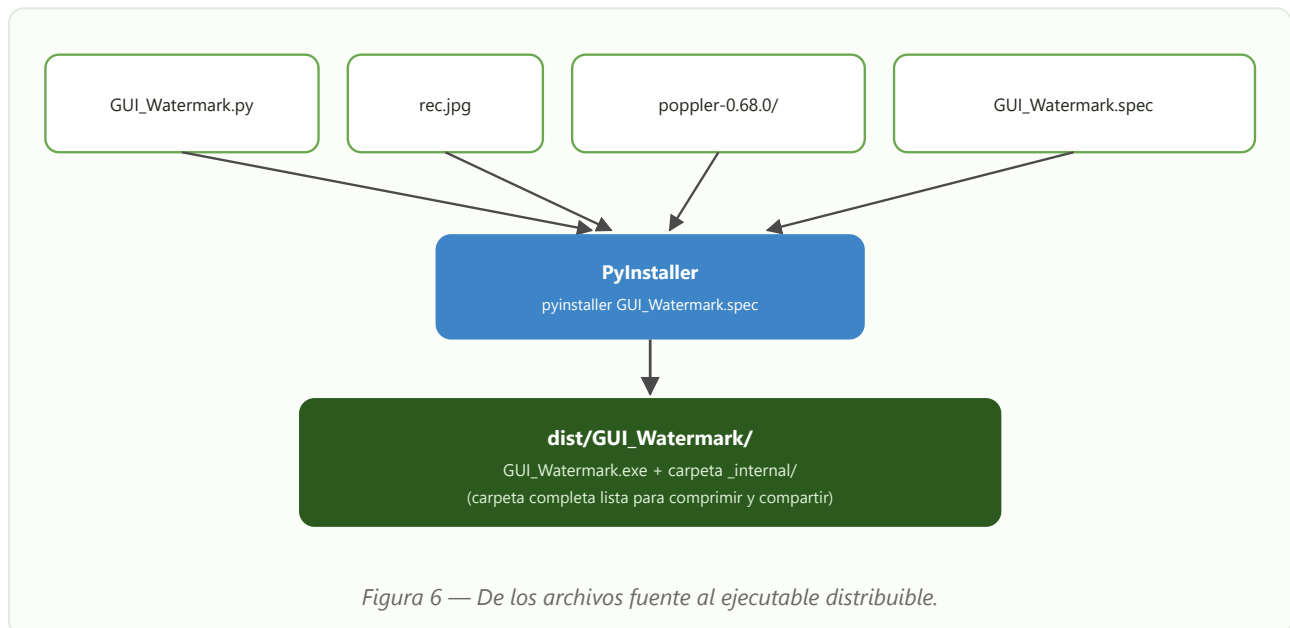
```
def main():
    app = QApplication(sys.argv)
    window = DragDropWindow()
    window.show()
    sys.exit(app.exec())

if __name__ == "__main__":
    main()
```

`QApplication` administra el bucle de eventos y los recursos gráficos de toda la aplicación (solo puede existir una instancia). Se crea la ventana, se muestra con `.show()` y `app.exec()` arranca el bucle de eventos hasta que la ventana se cierra.

14. Empaquetado a `.exe` (PyInstaller)

`GUI_Watermark.spec` define cómo PyInstaller construye el ejecutable, incluyendo los recursos que deben empaquetarse dentro de él:



```
a = Analysis(
    ['GUI_Watermark.py'],
    datas=[
        ('rec.jpg', '.'),
        ('poppler-0.68.0', 'poppler-0.68.0'),
    ],
    ...
)
exe = EXE(..., exclude_binaries=True, name='GUI_Watermark', console=False, ...)
coll = COLLECT(exe, a.binaries, a.datas, ..., name='GUI_Watermark')
```

`datas` le indica a PyInstaller que copie `rec.jpg` y toda la carpeta `poppler-0.68.0` dentro del paquete final; `get_base_path()` las localiza en tiempo de ejecución mediante `sys._MEIPASS`. `console=False` evita que se abra una consola detrás de la interfaz gráfica.

15. Solución de problemas

Síntoma	Causa probable	Solución
---------	----------------	----------

El <code>.exe</code> no abre / cierra inmediatamente.	Se movió o copió solo el <code>.exe</code> , sin la carpeta <code>_internal</code> .	Vuelve a descomprimir/copiar la carpeta <code>GUI_Watermark</code> completa.
Error al procesar un PDF específico dentro de un lote.	El PDF está dañado, cifrado o es un archivo no válido.	El resto del lote se procesa igualmente; revisa el mensaje de error mostrado al final para identificar el archivo.
La marca de agua se ve muy tenue o muy fuerte.	Valor de opacidad inadecuado, o la imagen ya tiene transparencia propia.	Ajusta el slider de opacidad (0–255) antes de procesar.
El proceso tarda mucho con muchos PDFs o PDFs muy largos.	Cada página se renderiza a <code>RENDER_DPI</code> (200 DPI) de forma individual; es un proceso intensivo en CPU.	Es esperable en lotes grandes; se puede cancelar en cualquier momento con el botón "Cancelar" de la barra de progreso.